

多场景下无人机烟幕干扰弹投放决策模型的构建与求解

摘要

针对空地导弹威胁下圆柱形真目标的烟幕遮蔽防护问题，本文构建了无人机-烟幕弹-导弹的协同建模与优化体系，旨在通过设计无人机飞行及烟幕弹投放策略，最大化真目标的有效遮蔽时长。

针对问题一，通过建立导弹、无人机与烟幕弹的运动学模型和几何遮蔽判定模型，构建描述进入与离开遮蔽时刻的非线性方程组，采用 **Newton-Raphson 方法**求解方程组和 **Halley 方法**进行数值求解，得到有效遮蔽时长为 1.44 秒。

针对问题二，采用**多阶段优化算法**求解无人机的最优烟幕弹投放策略。通过构建遮蔽效果模型，分析烟幕与导弹视线的动态关系，精确计算有效遮蔽时长。优化在航向、速度、投放时间和引信延时四个维度上进行，最终得到最优方案：无人机以 70 m/s 速度、 177° 航向飞行，起飞 1 秒投放，引信延时 3 秒。该方案实现遮蔽 4.724 秒。

针对问题三，基于导弹-目标-无人机-烟幕的一体化模型，采用动态云团模拟和可见性判断，生成遮蔽状态。通过枚举起爆方案并结合优化算法进行搜索调优，再经**蒙特卡洛**^[1] 扰动测试评估鲁棒性。得到最终方案：多弹遮蔽总时长达 6.660 秒。

针对问题四，本文研究了使用三架无人机协同干扰导弹 M1 的烟幕投放策略优化问题。该问题属于**高维非线性优化**范畴，通过建立几何约束和时序协调模型，采用混合智能算法进行求解。算法融合了多种优化技术，通过并行搜索和局部精细化策略确保了解的质量和稳定性。最终获得最优遮蔽时长为 13.19 秒。^[9]

针对问题五，采用基于**遗传算法**的协同优化策略，对多架无人机集群的航向、速度及多枚烟幕弹投放参数（时序与起爆高度）进行联合求解，有效克服了时空约束与弹目交会条件的复杂性。得到最终三枚导弹总共被有效遮蔽 10.650s。

关键词：遮蔽时长优化；多阶段优化算法；混合智能算法；遗传算法；蒙特卡洛测试；投放策略优化

1 问题重述

1.1 问题背景

在现代的对抗场景中，烟幕干扰弹凭借其成本低、效果佳的优势，成为干扰敌方导弹、保护己方目标的重要手段。其核心原理为通过化学燃烧或爆炸形成烟幕或气溶胶云团，在己方目标前方的特定空域形成遮蔽，阻挡敌方导弹对目标的探测以及锁定。随着技术的发展，无人机已经成为烟幕干扰弹的重要搭载平台——具备长续航的无人机可在特定空域巡飞，受领任务后能够精准控制干扰弹的投放及起爆，实现对己方目标的高效保护。^[7]

在实际应用场景中，为使得烟幕干扰效果更为有效，需要合理设计无人机投放干扰弹的策略，如无人机飞行方向、速度，干扰弹投放点位置、起爆点位置等。本研究即围绕着不同场景下（无人机的数量、无人机投放干扰弹的数量、敌方导弹的数量）无人机投放干扰弹的最优策略而展开。

1.2 问题描述

本任务聚焦“无人机投放烟幕干扰弹保护真目标”这一核心场景。以假目标为诱饵，敌方 3 枚空地导弹直奔假目标，需要通过 5 架预设初始位置的无人机 FY1-FY5 投放烟幕干扰弹，在敌方导弹与真目标之间形成遮蔽。

问题一：仅使用 FY1，保持以 120 m/s 的速度朝假目标方向等高以直线飞行。自受令 1.5 s 后投放 1 枚烟幕干扰弹，并设定 3.6 s 后起爆。求在此条件下，该弹对导弹 M1 的有效遮蔽时长。

问题二：仍只使用 FY1 与 1 枚烟幕干扰弹，通过选择 FY1 的航向、巡航速度和烟幕干扰弹的投放点、起爆点等参数，最大化对 M1 的有效遮蔽时长，并给出对应的最优参数与策略。

问题三：动用 FY1 与 3 枚烟幕干扰弹对 M1 实施干扰，设计三枚干扰弹的投放与起爆时空方案，并将结果整理到文件 result1.xlsx。

问题四：动用 FY1、FY2、FY3 三架无人机，各投放 1 枚烟幕干扰弹对 M1 进行干扰，设计三机的航向与速度及各弹的投放与起爆方案，以可能延长有效遮蔽时间，并将结果整理到文件 result2.xlsx。

问题五：出动 5 架无人机，每架至多投放 3 枚烟幕干扰弹，同时干扰 M1、M2、M3 等 3 枚来袭导弹，综合分配载弹与投放时空策略，尽可能增加对目标的有效遮蔽时长，并将结果整理到文件 result3.xlsx。

2 问题分析

2.1 问题一的分析

问题一已知无人机 FY1 的航向和巡航速度，以及 1 枚烟幕干扰弹的投放点和爆炸点，要求得到干扰弹的遮蔽时长，这属于**多主体运动学与几何分析相结合的时空碰撞判定问题**。研究这一问题可以为后续多机、多弹协同设计提供基准，验证特定参数组合下的策略是否合理。解决问题一的流程如图 1 所示。

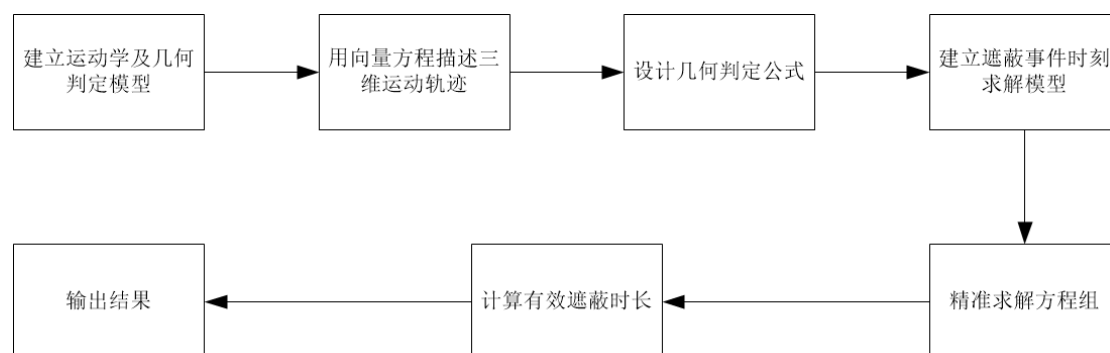


图 1: 解决问题一的流程图

2.2 问题二的分析

问题二要求自定义无人机 FY1 的航向和速度，以及 1 枚烟幕干扰弹的投放点和爆炸点，通过合理的方案使得遮蔽时间尽可能长，这属于典型的**多参数非线性优化类数学问题**^[2]。解决问题二的流程如图 2 所示。

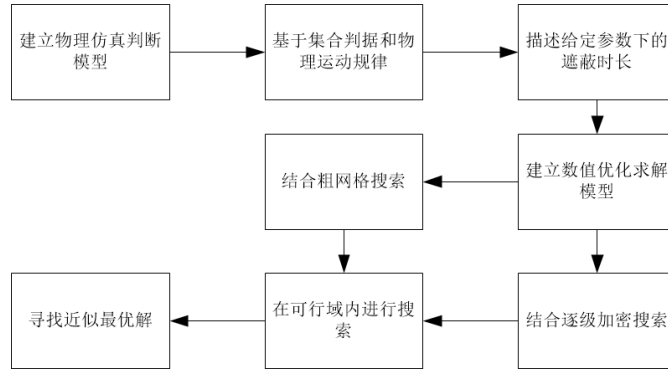


图 2: 解决问题二的流程图

2.3 问题三的分析

问题三要求自定义无人机 FY1 的航向和速度，以及 3 枚烟幕干扰弹的投放点和爆炸点，通过合理的方案使得遮蔽时间尽可能长，这属于典型的**约束优化问题**^[3]。解决问题三的流程如图 3 所示。

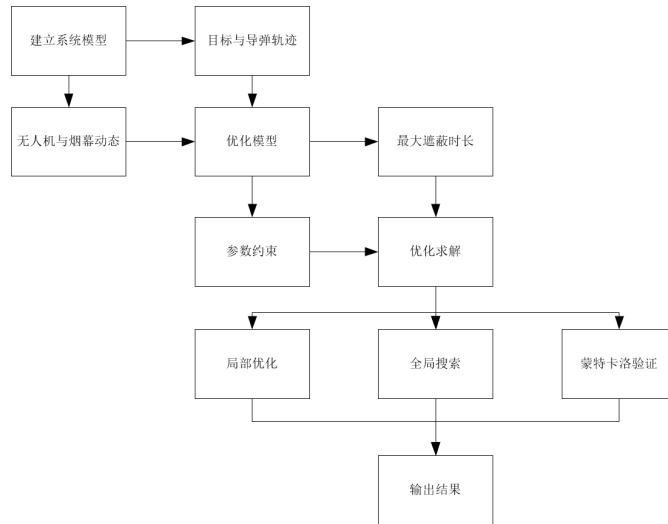


图 3: 解决问题三的流程圖

2.4 问题四的分析

问题四属于动态优化与轨迹规划的组合优化问题，通常采用分层优化或序列决策方法求解。数据显示三架无人机空间分布分散，高度差异明显，而导弹 M1 轨迹确定性强，为预测提供基础。问题要求最大化遮蔽时间，需在飞行路径、投放时机和起爆高度间权衡。解决问题四的流程如图 4 所示。

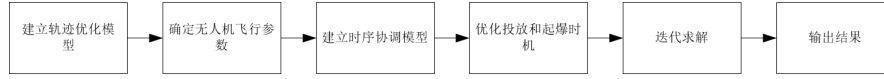


图 4: 解决问题四的流程图

2.5 问题五的分析

问题五要求用 5 架无人机对 3 枚来袭导弹进行干扰，核心是通过优化无人机飞行参数及烟幕弹投放、起爆点，最大化真目标有效遮蔽总时长，属于**多约束多目标的全局优化问题**^[4]。解决问题五的流程如图 5 所示。

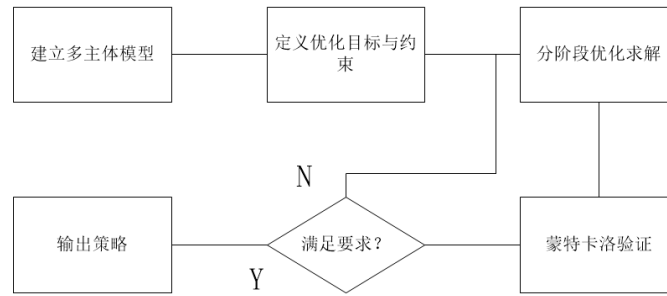


图 5: 解决问题五的流程图

3 模型假设与符号说明

3.1 模型假设

1. 无人机投放干扰弹前瞬时调整方向，投放后沿水平直线匀速飞行（高度不变）。
2. 干扰弹脱离无人机后，运动分为两段：
 - (1) 投放后至起爆前：干扰弹水平匀速（与无人机同速）+ 竖直自由落体；
 - (2) 起爆后：烟幕云团为水平位置固定+ 竖直匀速下沉。
3. 导弹沿直线匀速飞行，方向始终指向假目标。
4. 有效遮蔽判定标准：导弹视线（即导弹位置与真目标位置之间连线的连线）与烟幕云团存在交集，即导弹视线到烟幕球心的最小距离 10 m。
5. 在计算烟幕云团与真目标的空间关系时，将真目标看作质点，位置与真目标几何中心的位置作相同。

3.2 符号说明

表 1: 主要数学符号说明

符号	含义
O	假目标（坐标原点）位置向量
T	真目标位置向量
M_0	导弹初始位置向量
$M(t)$	导弹在时刻 t 的位置向量
U_0	无人机初始位置向量
$U(t)$	无人机在时刻 t 的位置向量
g	重力加速度
s^*	视线上最近点参数
tol	数值方法收敛容差
max_iter	最大迭代次数

4 模型建立与求解

4.1 问题一模型的建立与求解

4.1.1 模型的建立

1. 运动学及几何判定模型的建立

(1) 坐标系以及核心实体参数定义

以假目标为坐标原点 O 建立三维直角坐标系，水平面为 xOy 平面，竖直向上为 z 轴正方向。则真目标的位置（圆柱体的几何中心）为 T ，无人机 FY1 的初始位置为 $FY1_0$ ，导弹 M1 的初始位置为 $M1_0$ 。

(2) 关键物理与时序参数定义

在模型的建立过程中，关键物理与时序参数的定义如表 2 所示。

表 2: 关键物理与时序参数定义

参数类型	数学符号	数值/物理意义
运动参数	v_{UAV}	120 m/s, 无人机水平匀速飞行速度
	v_M	300 m/s, 导弹匀速飞行速度
	v_s	3 m/s, 烟幕云团球心匀速下沉速度
烟幕参数	R	10 m, 烟幕有效遮蔽半径
	ΔT	20 s, 烟幕起爆后有效时长
时序参数	t_{rel}	1.5 s, 无人机受领任务后到投放干扰弹的时间
	t_{fuse}	3.6 s, 烟幕弹引信延时
	t_0	$t_0 = t_{\text{rel}} + t_{\text{fuse}} = 5.1$ s, 干扰弹起爆时刻

2. 用向量方程描述三维运动轨迹

(1) 无人机投放点轨迹建模

无人机沿 x 轴负方向（朝假目标）水平匀速飞行，投放时刻 $t = t_{\text{rel}}$ 的位置即为烟幕弹投放点 P_{rel} ，其向量方程为

$$\mathbf{P}_{\text{rel}} = \mathbf{FY}\mathbf{1}_0 + v_{\text{UAV}} \cdot t_{\text{rel}} \cdot \mathbf{e}_x^-, \quad (1)$$

其中 $\mathbf{e}_x^- = (-1, 0, 0)$ 为 x 轴负方向单位向量。

(2) 干扰弹起爆点轨迹建模

烟幕弹从投放点脱离后，至起爆前（时长 t_{fuse} ）做“水平匀速+竖直自由落体”运动。在水平方向上，速度与无人机一致，水平位移向量

$$\Delta \mathbf{s}_h = v_{\text{UAV}} \cdot t_{\text{fuse}} \cdot \mathbf{e}_x^-; \quad (2)$$

竖直方向上，干扰弹自由落体位移向量

$$\Delta \mathbf{z} = -\frac{1}{2}gt_{\text{fuse}}^2 \cdot \mathbf{e}_z, \quad (3)$$

其中 $\mathbf{e}_z = (0, 0, 1)$ 为 z 轴正方向单位向量，负号表示向下。

起爆点 $\mathbf{C}_0$ （烟幕云团初始球心）的向量方程为

$$\mathbf{C}_0 = \mathbf{P}_{\text{rel}} + \Delta \mathbf{s}_h + \Delta \mathbf{z}. \quad (4)$$

(3) 烟幕云团球心轨迹建模

起爆后 ($t \geq t_0$), 烟幕云团仅沿 z 轴负方向匀速下沉, 任意时刻 t 的球心位置 $\mathbf{C}(t)$ 向量方程为

$$\mathbf{C}(t) = \mathbf{C}_0 - v_s \cdot (t - t_0) \cdot \mathbf{e}_z, \quad (5)$$

其中, $t - t_0$ 为起爆后时长, 下沉距离随时间线性增加, 水平坐标 (x, y) 与起爆点一致。

(4) 导弹轨迹建模

任意时刻 t 的导弹位置 $\mathbf{M}(t)$ 向量方程为

$$\mathbf{M}(t) = \mathbf{M}\mathbf{1}_0 + v_M \cdot t \cdot \mathbf{u}. \quad (6)$$

3. 设计几何判定公式 (遮蔽有效性判定)

明确核心判定标准为: 导弹视线 ($\mathbf{M}(t)$ 到 $\mathbf{T}$ 的线段) 与烟幕云团 (半径 R 的球体) 存在交集, 等价于 “视线到烟幕球心的最小距离 $d_{\min} \leq R$ ”。

(1) 视线线段参数化

将导弹视线 ($\mathbf{M}(t)$ 到 $\mathbf{T}$ 的线段) 用参数 $s \in [0, 1]$ 表示:

$$\mathbf{W}(s) = \mathbf{M}(t) + s \cdot (\mathbf{T} - \mathbf{M}(t)). \quad (7)$$

当 $s = 0$ 时, 对应导弹位置 $\mathbf{M}(t)$; $s = 1$ 时, 对应真目标位置 $\mathbf{T}$; $s \in (0, 1)$ 时, 对应视线线段中间任意点。

(2) 最小距离 $d_{\min}$ 计算

设烟幕球心 $\mathbf{C}(t)$, 定义向量

$$\mathbf{U} = \mathbf{C}(t) - \mathbf{M}(t), \quad (8)$$

$$\mathbf{V} = \mathbf{T} - \mathbf{M}(t). \quad (9)$$

下面分三种情形进行推导:

第一种情形：最近点在导弹端 ($s^* \leq 0$)。由向量投影公式得到线段参数

$$s^* = \frac{\mathbf{U} \cdot \mathbf{V}}{|\mathbf{V}|^2}, \quad (10)$$

若 $s^* \leq 0$ ，最近点为 $\mathbf{M}(t)$ ，于是

$$d_{\min} = |\mathbf{M}(t) - \mathbf{C}(t)| = |\mathbf{U}|. \quad (11)$$

第二种情形：最近点在真目标端 ($s^* \geq 1$)。若 $s^* \geq 1$ ，最近点为 $\mathbf{T}$ ，则有

$$d_{\min} = |\mathbf{T} - \mathbf{C}(t)|. \quad (12)$$

第三种情形：最近点在线段中间 ($0 < s^* < 1$)。由向量投影的垂直性质，得到最小距离

$$d_{\min} = \sqrt{|\mathbf{U}|^2 - \left(\frac{\mathbf{U} \cdot \mathbf{V}}{|\mathbf{V}|} \right)^2}. \quad (13)$$

(3) 遮蔽有效性判定公式

当满足以下两个条件时，判定为“有效遮蔽”：

$$\begin{cases} d_{\min} \leq R, \\ t \in [t_0, t_0 + \Delta T]. \end{cases} \quad (14)$$

4. 建立遮蔽事件时刻求解模型

首先要对两个关键时间点进行定义：进入时刻 t_{in} 以及离开时刻 t_{out} 。

进入时刻 t_{in} 是指首次满足 $d_{\min} = R$ (遮蔽开始) 的时刻，此时最近点在线段中间 ($0 < s < 1$)；离开时刻 t_{out} 是指末次满足 $d_{\min} = R$ (遮蔽结束) 的时刻，此时最近点在导弹端 ($s \leq 0$)。

(1) 进入时刻 t_{in} 的求解模型

因 $0 < s < 1$ ，需同时满足“距离等于半径”和“最近点在线段上”，建立二维非线性方程组

$$\begin{cases} F_1(t, s) = 0, \\ F_2(t, s) = 0. \end{cases} \quad (15)$$

由距离平方等于半径平方以及最近点与线段方向垂直，向量点积为 0 有

$$F_1(t, s) = |\mathbf{W}(s) - \mathbf{C}(t)|^2 - R^2 = 0, \quad (16)$$

$$F_2(t, s) = (\mathbf{W}(s) - \mathbf{C}(t)) \cdot \mathbf{V} = 0. \quad (17)$$

将其代入 (6)(7) 式展开得

$$F_1 = |\mathbf{M}(t) + s\mathbf{V} - \mathbf{C}(t)|^2 - R^2, \quad (18)$$

$$F_2 = (\mathbf{M}(t) + s\mathbf{V} - \mathbf{C}(t)) \cdot \mathbf{V}. \quad (19)$$

(2) 离开时刻 t_{out} 的求解模型

因 $s \leq 0$ ，最近点为导弹位置 $\mathbf{M}(t)$ ，建立单变量方程

$$g(t) = |\mathbf{M}(t) - \mathbf{C}(t)|^2 - R^2 = 0, \quad (20)$$

代入 $\mathbf{M}(t)$ 和 $\mathbf{C}(t)$ 的轨迹方程，展开得

$$g(t) = |\mathbf{M}\mathbf{1}_0 + v_M t \mathbf{u} - (\mathbf{C}_0 - v_s(t - t_0)\mathbf{e}_z)|^2 - R^2. \quad (21)$$

5. 计算有效遮蔽时长

(1) 有效遮蔽时长计算

有效遮蔽时长为离开时刻与进入时刻的差值，需满足 $t_{\text{out}} > t_{\text{in}}$ 且 $t_{\text{out}} \leq t_0 + \Delta T$ ，公式为

$$T_{\text{shield}} = \max(0, \min(t_{\text{out}}, t_0 + \Delta T) - t_{\text{in}}). \quad (22)$$

(2) 一致性校核

计算 t_{in} 时视线最近点到烟幕球心的距离，应满足

$$|\mathbf{W}(s_{\text{in}}) - \mathbf{C}(t_{\text{in}})| \approx R. \quad (23)$$

计算 t_{out} 时导弹到烟幕球心的距离，应满足

$$|\mathbf{M}(t_{\text{out}}) - \mathbf{C}(t_{\text{out}})| \approx R. \quad (24)$$

4.1.2 模型的求解

1. 求解思路

由于方程组 (15) 和 (20) 均为非线性方程，无法直接解析求解，因此采用数值方法进行高精度近似。具体地：

(1) 对于进入时刻 t_{in} 的二维方程组，采用 **Newton-Raphson 方法**^[5] 进行迭代求解；

(2) 对于离开时刻 t_{out} 的一维方程，采用 **Halley 方法** 以提高收敛速度和精度。

2. 参数设置与初始值选取

根据表 1，可确定各参数的具体取值；根据建立的空间直角坐标系，可以得到初始场景中各实体位置：假目标 $O(0, 0, 0)$ ，真目标 $T(0, 200, 0)$ ， $\mathbf{FY1}_0 = (10000, 0, 1000)$ ， $\mathbf{M1}_0 = (20000, 0, 5000)$ 。初始场景布局如图 6 所示：

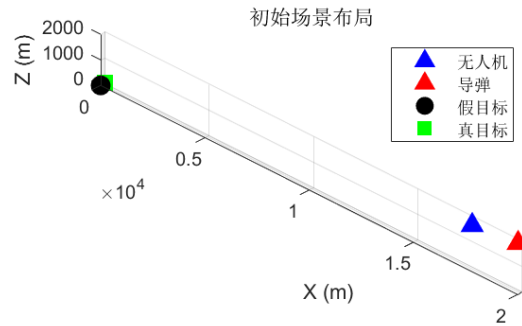


图 6: 初始场景布局

3. 进入时刻 t_{in} 的求解

使用 Newton-Raphson 方法求解方程组 (15)，选取初始猜测值 $t^{(0)} = 8.0 \text{ s}$ ， $s^{(0)} = 0.5$ ，经过 4 次迭代后收敛，得到 $t_{\text{in}} = 8.01 \text{ s}$ ， $s_{\text{in}} = 0.02$ ，收敛过程如图 7 所示。

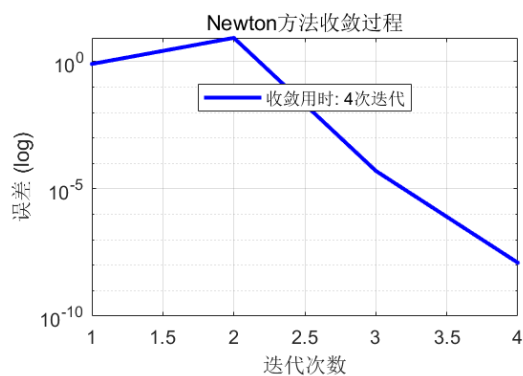


图 7: Newton 方法收敛过程

进入时刻的几何关系如图 8 所示，可见视线与烟幕球首次相切。

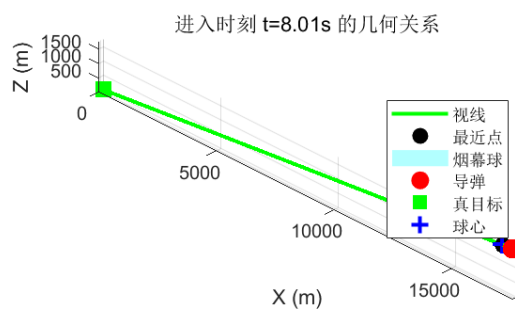


图 8: 进入时刻各主体的几何关系

4. 离开时刻 t_{out} 的求解

使用 Halley 方法求解方程 (20)，初始猜测值 $t^{(0)} = 9.0$ s，经 4 次迭代后收敛，得到 $t_{out} = 9.45$ s。收敛过程如图 9 所示。

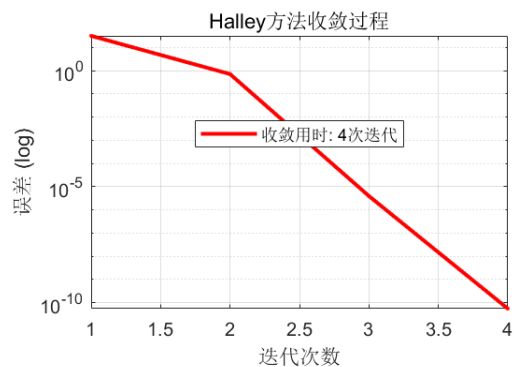


图 9: Halley 方法收敛过程

离开时刻的几何关系如图 10 所示，可见导弹与烟幕球相切。

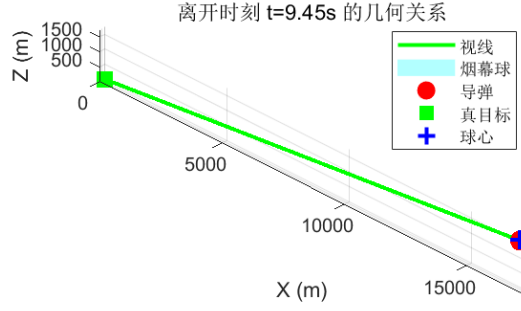


图 10: 离开时刻各主体的几何关系

5. 有效遮蔽时长计算

根据 (22) 式，代入数值得到 $T_{\text{shied}} = 1.44 \text{ s}$ 。最小距离 $d_{\min}(t)$ 随时间变化曲线如图 11 所示，可见在 $t \in [8.01, 9.45]$ 区间内， $d_{\min} \leq R = 10 \text{ m}$ ，且烟幕处于有效期内 ($t \leq 25.1 \text{ s}$)，验证了遮蔽时长的正确性。

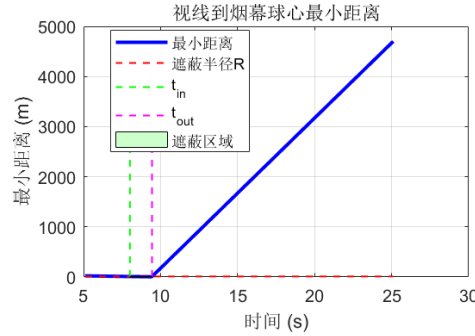


图 11: 导弹视线到烟幕球心最小距离变化图

4.2 问题二模型的建立与求解

4.2.1 模型的建立

1. 物理仿真判定模型的建立

在问题二中，首要任务是“把‘烟幕遮蔽’的过程构造成一个严格的数学模型，以便能够量化遮蔽时长并进行优化。首先，导弹以恒定速度沿着指向假目标的方向直线飞行。

无人机的轨迹则由航向角 θ 、恒速 v 投放时刻 t_{rel} 和引信延时 t_{fuse} 四个控

制变量决定。无人机的水平飞行方向为

$$\mathbf{d}(\theta) = (\cos \theta, \sin \theta, 0), \quad (25)$$

在投放时刻，其位置为

$$\mathbf{P}_{\text{rel}} = F\mathbf{Y}_1 + vt_{\text{rel}}\mathbf{d}(\theta). \quad (26)$$

干扰弹在释放后经过延时 t_{fuse} 起爆。在延时阶段，它同时向前运动并受到重力作用竖直下落，则起爆点坐标为

$$\mathbf{P}_{\text{burst}} = \mathbf{P}_{\text{rel}} + vt_{\text{fuse}}\mathbf{d}(\theta) - \frac{1}{2}gt_{\text{fuse}}^2\mathbf{e}_z, \quad \mathbf{e}_z = (0, 0, 1). \quad (27)$$

起爆时刻则是

$$t_{\text{burst}} = t_{\text{rel}} + t_{\text{fuse}}. \quad (28)$$

在此之后，烟幕被视为一个半径为 R 的球体，自 $\mathbf{P}_{\text{burst}}$ 处开始以匀速 v_s 向下沉降，则球心轨迹为

$$\mathbf{C}(t) = \mathbf{P}_{\text{burst}} - (0, 0, v_s(t - t_{\text{burst}})), \quad t \geq t_{\text{burst}}. \quad (29)$$

为了判定是否遮蔽，需要计算云团中心与“导弹位置-真目标”视线段的最近距离。设真目标位置为 $\mathbf{T}$ ，定义

$$s = \frac{(\mathbf{C}(t) - \mathbf{M}(t)) \cdot (\mathbf{T} - \mathbf{M}(t))}{\|\mathbf{T} - \mathbf{M}(t)\|^2}. \quad (30)$$

根据 s 的取值不同，最小距离可写为

$$d_{\min}(t) = \begin{cases} \|\mathbf{C}(t) - \mathbf{M}(t)\|, & s \leq 0, \\ \|\mathbf{C}(t) - \mathbf{T}\|, & s \geq 1, \\ \|\mathbf{C}(t) - (\mathbf{M}(t) + s(\mathbf{T} - \mathbf{M}(t)))\|, & 0 < s < 1. \end{cases} \quad (31)$$

当存在区间满足 $d_{\min}(t) \leq R$ 时，可以认为导弹的视线被遮断，遮蔽时长则由进入和退出这一条件的时刻差给出，即 $t_{\text{shield}} = t_{\text{exit}} - t_{\text{enter}}$ 。

综上所述，问题二可归结为一个以遮蔽时长为目标函数的优化问题：

$$\max_{\theta, v, t_{\text{rel}}, t_{\text{fuse}}} t_{\text{shield}}(\theta, v, t_{\text{rel}}, t_{\text{fuse}}), \quad (32)$$

同时受到飞行速度、投放时机和引信延时的取值范围约束。该模型完整刻画了导弹、无人机与烟幕干扰弹三者之间的动态关系，并为后续搜索与优化提供了理论基础。

2. 数值优化求解模型的建立

在这里，将目标函数写成其负数，以适配标准优化器最小化的习惯：

$$\max_{\theta, v, t_{\text{rel}}, t_{\text{fuse}}} -t_{\text{shield}}(\theta, v, t_{\text{rel}}, t_{\text{fuse}}). \quad (33)$$

优化涉及四个连续决策变量：

航向角 $\theta \in [\theta_{\min}, \theta_{\max}]$ ，无人机飞行速度 $v \in [v_{\min}, v_{\max}]$ ，投放时刻 $t_{\text{rel}} \in [0, T_{\max}]$ ，引信延时 $t_{\text{fuse}} \in [t_{\min}, t_{\max}]$ 。因此优化模型具有有界连续约束。

除此之外，还存在物理约束，整合为 $g_i(\theta, v, t_{\text{rel}}, t_{\text{fuse}}) \leq 0$ 。

综合上述，数值优化求解模型可以表示为

$$\left\{ \begin{array}{l} \min_{\theta, v, t_{\text{rel}}, t_{\text{fuse}}} -f(\theta, v, t_{\text{rel}}, t_{\text{fuse}}), \\ \theta_{\min} \leq \theta \leq \theta_{\max}, \\ v_{\min} \leq v \leq v_{\max}, \\ t_{\min}^{\text{rel}} \leq t_{\text{rel}} \leq t_{\max}^{\text{rel}}, \\ t_{\min}^{\text{fuse}} \leq t_{\text{fuse}} \leq t_{\max}^{\text{fuse}}, \\ g_i(\theta, v, t_{\text{rel}}, t_{\text{fuse}}) \leq 0, \quad i = 1, 2, \dots \end{array} \right. \quad (34)$$

4.2.2 模型的求解

1. 求解思路

烟幕遮蔽优化属于复杂的非线性问题，由于目标函数涉及导弹轨迹、烟幕运动和几何遮蔽关系的动态耦合，直接采用解析方法几乎不可能，因此必须借助数值方法进行求解。

2. 求解过程

求解的基础是建立一个精确的遮蔽时长评估机制。首先给定一组决策变量 $(\theta, v, t_{\text{rel}}, t_{\text{fuse}})$ 后，通过时间离散化方法模拟整个遮蔽过程。算法以极小的时间增量 $\Delta t = 0.002$ 秒推进仿真时钟，在每个采样点上计算烟幕云团与导弹视线的相对位置关系。遮蔽判定的关键在于求解点到线段的最小距离问题——将烟幕中心向导弹-目标连线作垂直投影，通过向量内积运算确定投影点位置，进而计算最小距离 $d_{\min}$ 。只有当 $d_{\min} \leq R$ 且烟幕尚未消散时，才认定该时刻存在有效遮蔽。如图 12 所示，展现了烟幕遮蔽优化问题设置的场景，包含问题场景设置的三维图与俯视图（遮蔽半径示意）。

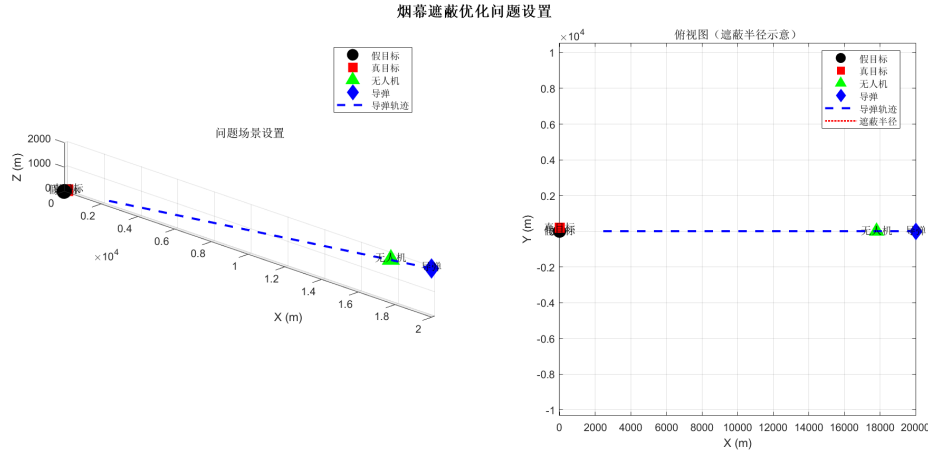


图 12: 烟幕遮蔽优化问题设置图

针对四维决策空间的搜索问题，算法采用了由粗到细的多分辨率优化框架。初始阶段通过大范围扫描确定搜索域。航向角集中在 170° - 180° 区间，速度和时间参数分别覆盖 70-140 m/s 和 0-15 秒。避免算法过早收敛到次优解。

在中等精度优化环节，搜索焦点将集中于前期粗筛所发现的高价值解邻域内。通过压缩参数范围并加密离散网格，该阶段显著提升了解的精度：航向角调整步长精细至 0.5° ，速度解析度提高到 1 m/s，时间参数的分辨率强化为 0.5 秒。

如图 13 所示，展现了优化过程分析的多个图表，包含优化收敛过程、航向角、飞行速度、投放时刻、引信延时的演化情况，以及各阶段改进统计，体现不同优化阶段（粗搜索、中等精度等）下相关参数的变化与改进次数分布。

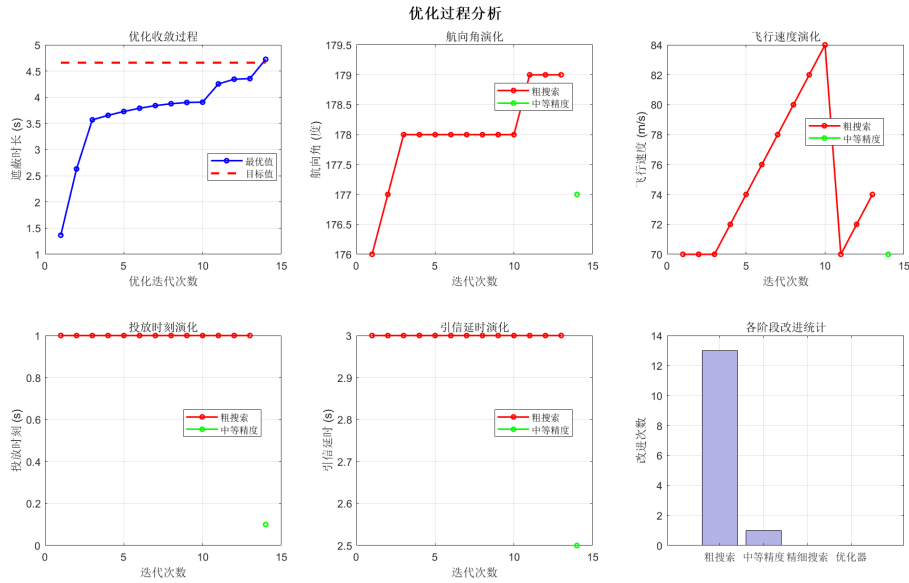


图 13: 优化分析图

当离散搜索达到极限后，算法转入基于梯度的连续优化阶段。该阶段以离散最优解为初值，借助 MATLAB 的 `fmincon` 求解器在连续域中进行精细化求解。采用**序列二次规划（SQP）方法**^[6]构造二次近似子问题，以快速逼近局部最优。为提高全局收敛能力，引入多起点策略，通过在全局最优解邻域采样生成多个初始点并行优化，最终选取最佳结果。

优化中需应对约束边界处的数值不稳定问题。接近可行域边界时，目标函数梯度易发生突变，引发振荡甚至发散。通过自适应约束松弛技术，在维持解可行性的前提下，容许边界附近有限试探，从而改善数值稳定性。

经四阶段递进优化，算法得到最优投放方案：无人机以 70.0 m/s 速度、 177.0° 航向飞行，于起飞后 1 秒投放，引信延时 3 秒起爆。该策略实现 4.7240 秒的有效遮蔽时长。

如图 14 所示，展现了综合分析（遮蔽时长：4.7240 s）的多个图表，包含 3D 轨迹、最小距离曲线、烟幕高度变化、XY 平面投影、速度分量对比以及关键时刻时间线，呈现出导弹、烟幕、无人机等在运动与遮蔽过程中的相关情况。

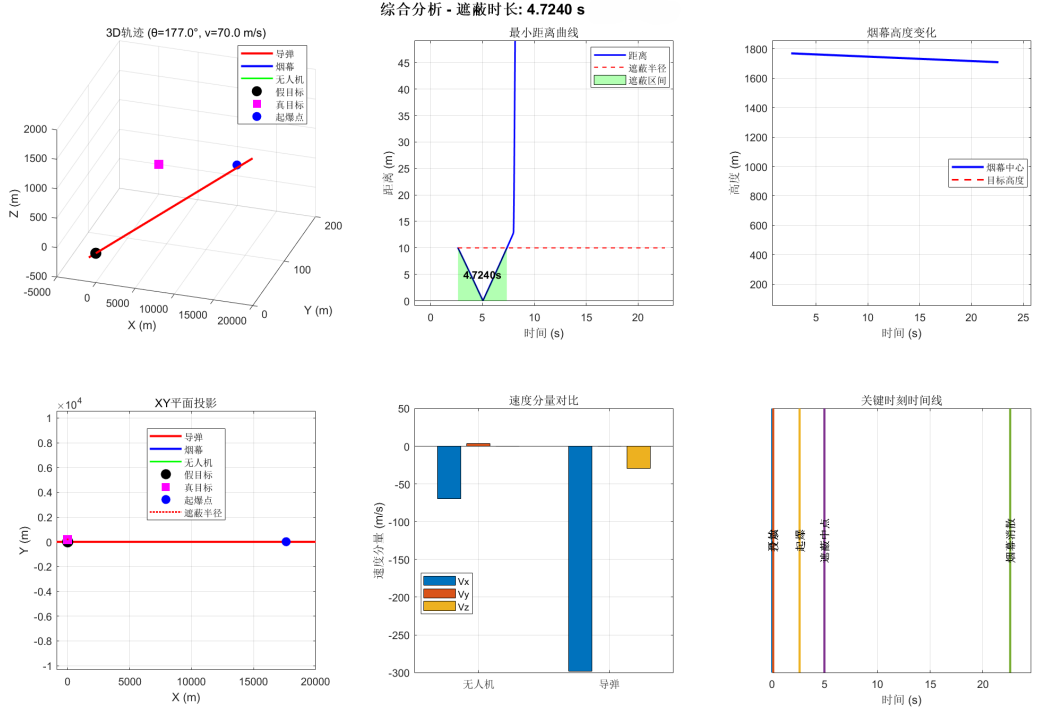


图 14: 综合分析-遮蔽时长图

4.3 问题三模型的建立与求解

4.3.1 模型的建立

无人机初始位置为 $U_0=(17800, 0, 1800)$ ，受领任务后，无人机瞬时确定航向并在恒定高度 U_{0z} 处以恒速直线飞行，航向用单位向量 $\mathbf{u}$ 表示，无人机飞行速度 $V_u \in [70, 140]$ m/s。因此

$$\mathbf{U}(t) = \mathbf{U}_0 + \mathbf{v}_u t \mathbf{u}, \quad (35)$$

$$U_z(t) = U_{0z}. \quad (36)$$

烟幕干扰弹从无人机释放后仅受重力作用，忽略空气阻力，其竖直自由落体时间与落高满足

$$t_{\text{fall}} = \sqrt{\frac{2(U_{0z} - H)}{g}}, \quad g = 9.8 \text{ m/s}^2, \quad (37)$$

其中 H 为起爆高度。

若第 i 枚干扰弹在 $t_{\text{drop},i}$ 释放、 $t_{\text{burst},i}$ 爆炸，则

$$t_{\text{burst},i} = t_{\text{drop},i} + t_{\text{fail},i}, \quad (38)$$

$$t_{\text{fail},i} = \sqrt{\frac{2(U_{0z} - H_i)}{g}}. \quad (39)$$

为使云团尽可能“对准”导弹与真目标之间的视线，令第 i 枚弹在起爆瞬间的水平位置位于无人机的水平投影处，即

$$C_{xy,i}(t_{\text{burst},i}) = U_{xy}(t_{\text{burst},i}). \quad (40)$$

其起爆高度 H_i 取为视线段 $\overline{M(t_{\text{burst},i})T}$ 上的线性插值高度。设

$$\Delta_{MT}(t) = T - M(t), \quad L^2(t) = \|\Delta_{MT}(t)\|^2, \quad (41)$$

$$s_i(t) = \text{clip}_{[0,1]} \left(\frac{(U_{xy}(t) - M_{xy}(t)) \cdot \Delta_{MT,xy}(t)}{\|\Delta_{MT,xy}(t)\|^2} \right), \quad (42)$$

则沿 $\overline{M(t_{\text{burst},i})T}$ 的高度内插为

$$H_i = M_z(t_{\text{burst},i}) + S_i(t_{\text{burst},i})(T_z - M_z(t_{\text{burst},i})) \quad (43)$$

据此由上式反算 $t_{\text{fail},i}$ 与 $t_{\text{drop},i}$ 。为保证起爆初期即具备遮蔽能力，要求无人机起爆时的水平位置与视线在水平面的最短距离不超过初始云团半径 r_0 ，即

$$\text{dist}(U_{xy}(t_{\text{burst},i}), M_{xy}(t_{\text{burst},i}) + S_i(t_{\text{burst},i})\Delta_{MT,xy}(t_{\text{burst},i})) < r_0. \quad (44)$$

根据试验描述，云团在起爆后瞬时形成球状，并以近似恒定的终端速度下沉。为更贴近真实扩散与沉降的过渡过程，本文采用“扩散 + 线性膨胀”的等效半径与“指数趋近终速”的沉降模型。

设年龄（自起爆的历时）为 $p = t - t_{\text{burst},i} \geq 0$ ，则第 i 枚云团的等效半径与中心高度分别为

$$r_i(p) = \min\{t_{\max}, \sqrt{r_0^2 + 2D_{xy}p + a_{lin}p}\}, \quad (45)$$

$$C_i(t) = H_i - V_{\text{term}}(1 - e^{-p/p_{\text{sink}}}), \quad (46)$$

其中 D_{xy} 为水平扩散系数, a_{lin} 为近似线性膨胀项, V_{term} 为终端下沉速度, p_{sink} 为趋近时间常数, $r_{\max}$ 为半径上限。

云团中心的空间坐标为

$$C_i(t) = (U_X(t_{\text{burst},i}), U_Y(t_{\text{burst},i}), Z_i(t - t_{\text{burst},i})), \quad t \geq t_{\text{burst},i}. \quad (47)$$

遮蔽判定采用“视线段与球面相交”的几何条件。对任意时刻 t , 将云团中心 $C_i(t)$ 正交投影到视线段 $\overline{M(t_{\text{burst},i})T}$ 上: 令

$$s^*(t) = \text{clip}_{[0,1]} \left(\frac{C_i(t) - M(t) \cdot \Delta_{MT}(t)}{L^2(t)} \right), \quad (48)$$

$$Q(t) = M(t) + s^*(t)\Delta_{MT}(t). \quad (49)$$

则第 i 枚云团在 t 时对视线的遮蔽判定为

$$\chi_i(t) = \mathbf{1}(\|C_i(t) - Q(t)\| \leq r_i(t - t_{\text{burst},i}), 0 \leq t - t_{\text{burst},i} \leq T_{\text{eff}}), \quad (50)$$

其中 $\mathbf{1}(\cdot)$ 为指示函数, 且仅在有效时窗 T_{eff} 内计入贡献。多枚干扰弹的总体遮蔽是逐时刻的“或”并集

$$\chi(t) = \max_i \chi_i(t), \quad (51)$$

总有效遮蔽时间作为目标函数记为

$$J = \int_0^{t_{\max}} \chi(t) dt \quad (52)$$

其中 $t_{\max} = \frac{\|O - M_0\|}{V_m} + t_{\text{margin}}$ 覆盖导弹飞抵假目标前的全过程 (附加少量冗余时域)。数值实现中采用均匀时间网格步长 Δt 离散化, 将积分替换为加权求和 $J \approx \sum_k X(t) \Delta t$ 。

约束方面, 释放一起爆的物理约束由上述重力关系给出, 高度限制 $H_i \in$

$[H_{\min}, H_{\max}]$, 无人机速度与航向在受领任务后固定不变且 $V_u \in [70, 140]$ 。单机多弹的最小释放间隔记为 $\Delta t_{\min}$, 即对任意相邻两弹有 $t_{\text{drop}, i+1} - t_{\text{drop}, i} \geq \Delta t_{\min}$

为避免无效候选, 起爆时无人机水平位置与视线段的水平距离须不超过初始半径 r_0 , 并在起爆后仅计入不超过 T_{eff} 的贡献。则问题可表述为

$$\max_{\theta, v_u, t_{\text{drop}}, t_{\text{fail}}} J(\theta, v_u, t_{\text{drop}}, t_{\text{fail}}), \quad \text{s.t.} \quad \begin{cases} v_u \in [70, 140], \\ t_{\text{fail}} = \sqrt{\frac{2(U_0 - H)}{g}}, \\ H \in [H_{\min}, H_{\max}]. \end{cases} \quad (53)$$

4.3.2 模型的求解

1. 求解思路

针对烟幕干扰弹投放策略的连续时空优化问题, 采用“离散化—候选枚举—组合优化—连续微调”的分层求解策略。

首先将连续的时间域离散化为细网格, 在每个时刻基于“线段—球体相交”几何判定构建遮蔽掩码, 将连续积分转化为离散求和。

外层在速度—航向空间进行致密扫描, 内层通过多分辨时间步长枚举起爆候选, 形成既致密又可控的候选库。

在候选库基础上, 将问题转化为离散组合优化, 使用遗传算法—坐标爬山—模拟退火的混合策略搜索最优投放方案, 最后通过连续微调与蒙特卡洛验证提升解的精度与鲁棒性。

图 15 展现了云团演化动态和物理模型, 包含云团 1、云团 2、云团 3 的半径与垂直下降距离随爆散后时间的变化, 它们的 3D 演化情况, 还给出了云团物理模型的参数 (如初始半径、最大半径等) 及相关数学模型。

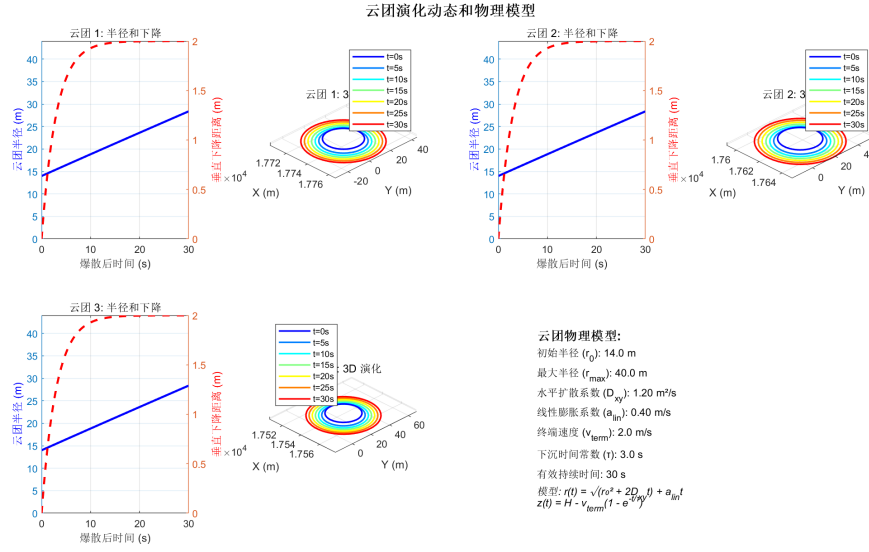


图 15: 三维场景与轨迹总览图

2. 求解过程

首先建立评估时间轴并预计算导弹与无人机的位姿序列，将云团抽象为“时变半径 + 指数沉降”的球体。

随后对每个时刻，将云团中心正交投影到导弹—真目标的视线段上，若中心到投影点的距离不超过当时半径且仍处于有效年龄，则判定该时刻被遮蔽；对多枚云团取时刻并集即可得到整段时间的遮蔽序列并累加求得时长。

如图 16 所示展现了优化景观和参数空间，包含优化景观的三维图与参数空间等高线图，呈现出无人机速度、偏航角等参数与目标函数的关联情况。

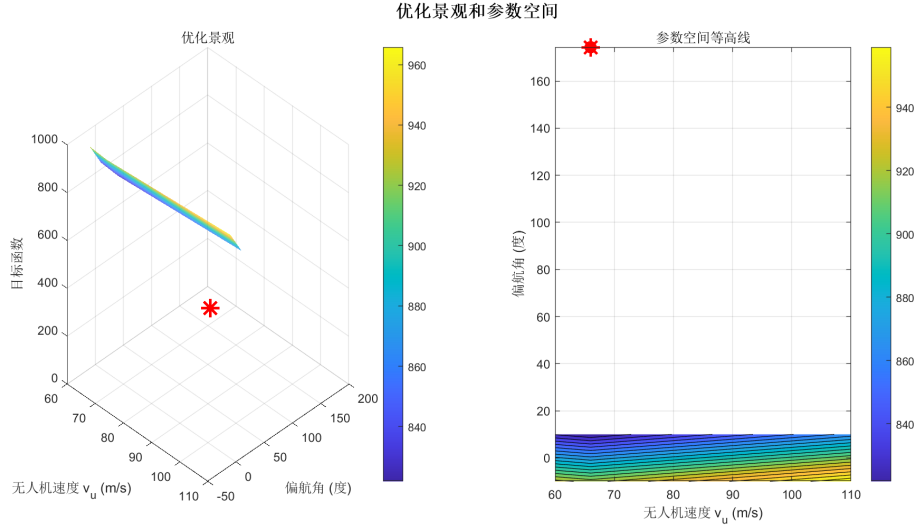


图 16: 遮蔽几何判定示意图

先在无人机速度与航向的外层网格扫描，内层用多分辨率时间步枚举起爆时刻；按视线几何确定起爆高度并由自由落体反推释放时刻，结合高度范围、起爆瞬时水平距离与有效时窗做可行性筛查；对可行解即时生成遮蔽掩码与贡献权重，并按起爆时间分箱仅保留高贡献者，形成精炼候选库。

随后将问题转为“从候选中选定固定枚数以最大化并集遮蔽时长”的离散组合优化，用光滑软惩罚纳入最小释放间隔与目标窗，采用遗传算法 + 少量坐标式爬山并配合温度衰减的退火接受以跳出局部最优，多随机起点并行求解后择优输出方案。

如图 17 所示，描绘了轨迹规划和拦截分析的相关图表，包含 3D 轨迹规划图（呈现导弹、无人机等的空间轨迹）、飞行器速度随时间变化图，以及展示角度和无人机 - 导弹距离的拦截几何图，体现出导弹、无人机等在轨迹规划与拦截过程中的运动及相关参数变化。

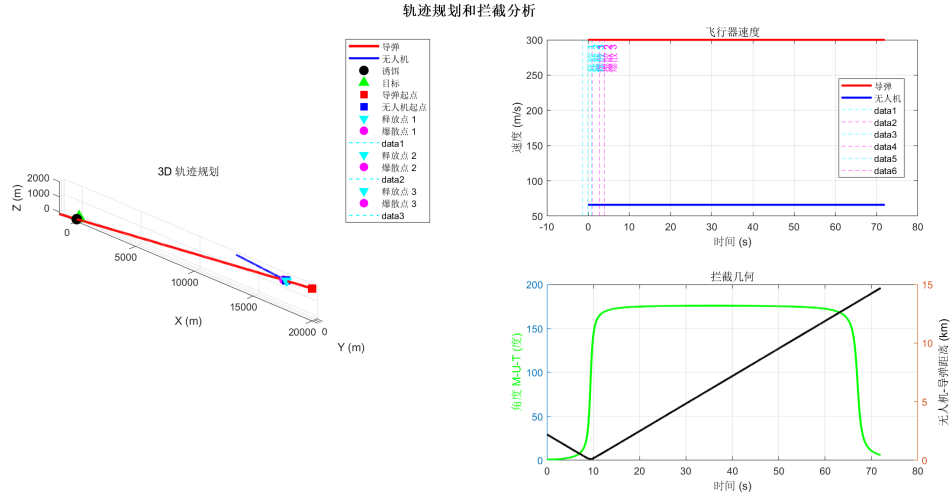


图 17: 遮蔽时间线与并集效果图

在得到离散最优组合后，对各起爆时刻在小窗口内作连续微调，联动更新起爆高度与释放时刻，仅重建该枚掩码并与其余掩码并集，从而以较小的计算代价进一步提升总遮蔽时长并将解压进目标窗。

对释放时刻、起爆高度、无人机速度与目标位置施加小幅高斯扰动，进行多次蒙特卡洛评估，统计成功率、均值与方差，用以检验方案在现实扰动下的稳健性与可实施性。

如图 18 所示，展现了覆盖分析和云团交互的相关图表，包含单个云团覆盖贡献、单个云团有效时间、覆盖重叠矩阵以及瞬时覆盖率等内容，呈现出云团 1、云团 2、云团 3 在覆盖方面的贡献、有效时间、重叠情况以及瞬时覆盖率的变化（含与目标最小值的对比）。

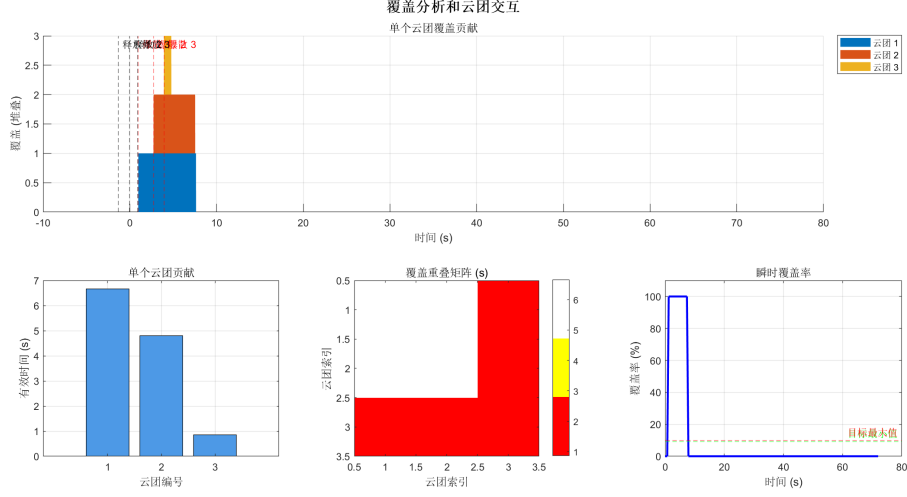


图 18: 混合全局搜索的收敛过程图

最终求得最优解，总遮蔽时长 6.660s。详细方案见支撑材料 result1.xlsx。

4.4 问题四模型的建立与求解

4.4.1 模型的建立

每架无人机以速度 $v_i \in [70, 140]$ m/s 匀速飞行，航向角为 θ_i 。无人机在时刻 $t_{0,i}$ 投放干扰弹，经时延 Δt_i 后于起爆点起爆。考虑重力作用，起爆点三维坐标为

$$(x_{b,i}, y_{b,i}, z_{b,i}) = \left(x_{0,i} + v_i \cos \theta_i \cdot t_{b,i}, y_{0,i} + v_i \sin \theta_i \cdot t_{b,i}, z_{0,i} - \frac{1}{2} g (\Delta t_i)^2 \right), \quad (54)$$

其中 $t_{b,i} = t_{0,i} + \Delta t_i$, $g = 9.8 \text{ m/s}^2$ 。

烟幕起爆后形成半径为 $R_s = 10 \text{ m}$ 的球状云团，以 $v_z = 3 \text{ m/s}$ 匀速下沉。起爆后经过时间 $\tau \in [0, 20]$ 秒，云团中心坐标为

$$C_i(\tau) = (x_{b,i}, y_{b,i}, z_{b,i} - v_z \tau). \quad (55)$$

云团对视线形成遮蔽的条件为：导弹与目标连线线段与球状云团相交。记 $L_0 = M(t) - C_i(t - t_{b,i})$ ，方向向量 $\mathbf{d} = T - M(t)$ ，则参数化射线方程为 $P(u) =$

$M(t) + u \cdot \mathbf{d}$, 代入球方程得到

$$\|P(u) - C_i(t - t_{b,i})\|^2 = R_s^2. \quad (56)$$

展开得二次方程 $au^2 + bu + c = 0$, 其中 $a = \|\vec{d}\|^2$, $b = 2\vec{d} \cdot L_0$, $c = \|L_0\|^2 - R_s^2$ 。

若判别式 $D \geq 0$ 且存在根 $u^* \in [0, 1]$, 则视线被遮蔽。

优化目标为最大化总有效遮蔽时间, 即

$$T_{\text{total}} = \int_0^{t_{\text{end}}} \mathbb{I} \left(\bigvee_{i=1}^3 S_i(t) \right) dt, \quad (57)$$

其中 $S_i(t)$ 表示云团 i 在时刻 t 是否有效遮蔽, $t_{\text{end}} = \frac{\|M_0\|}{v_m} \approx 66.67$ s。决策变量包括每架无人机的 θ_i 、 v_i 、 $t_{0,i}$ 及 Δt_i , 共 12 维, 需满足 $v_i \in [70, 140]$, $t_{0,i} \geq 0$, $\Delta t_i \geq 0$, 且 $t_{0,i} + \Delta t_i + 20 \leq t_{\text{end}}$ 。

该模型为高阶非线性优化问题, 采用混合优化策略对参数空间进行探索, 通过数值积分评估遮蔽时长, 最终解算出最优参数组合。

4.4.2 模型的求解

针对所建立的高维非线性优化模型, 我们采用混合智能优化策略进行求解。该策略融合了多种算法的优势, 首先生成多样化初始种群, 随后通过自适应岛模型框架并行运行粒子群优化与差分进化算法, 并引入精英保留与个体迁移机制确保优秀解的传递。

如图 19 所示, 展现了 PSO、JADE、GA、CMA - ES、SA、CD 等算法的收敛比较情况, 随着迭代次数增加, 各算法的适应度 (暴露时间) 均逐渐下降, 可直观对比不同算法的收敛趋势与效果。

算法收敛比较

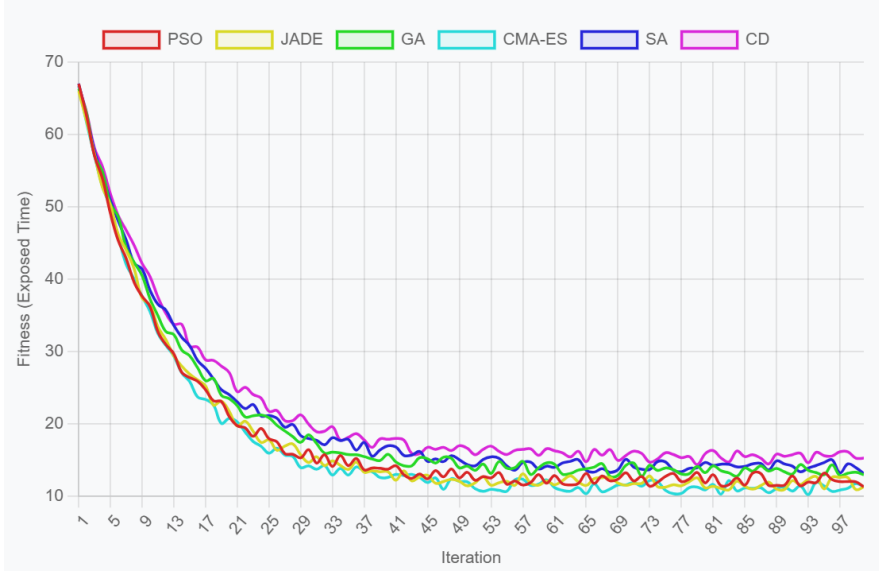


图 19: 算法收敛图

为进一步提升解的质量，我们集成了代理模型辅助筛选机制加速搜索过程，最后采用协同坐标下降与模拟退火等局部搜索技术进行精细调优。经过上述优化流程，最终得到总遮蔽时长为 13.19 秒的最优投放策略。具体方法见支撑材料 result2.xlsx。

4.5 问题五模型的建立与求解

4.5.1 模型的建立

1. 导弹飞行模型

设导弹 M_m ($m = 1, 2, 3$, 对应 M1、M2、M3) 初始位置为 (x_{m0}, y_{m0}, z_{m0}) ，假目标为原点 $(0, 0, 0)$ ，飞行速度 $v_m = 300 \text{ m/s}$ 。

导弹飞行方向向量为

$$\mathbf{d}_m = (-x_{m0}, -y_{m0}, -z_{m0}), \quad (58)$$

单位方向向量

$$\mathbf{u}_m = \frac{\mathbf{d}_m}{|\mathbf{d}_m|}, \quad (59)$$

其中 $|\mathbf{d}_m| = \sqrt{x_{m0}^2 + y_{m0}^2 + z_{m0}^2}$ 为导弹初始到假目标的直线距离。

导弹在 t 时刻的位置为

$$(x_m(t), y_m(t), z_m(t)) = (x_{m0}, y_{m0}, z_{m0}) + v_m \cdot t \cdot \mathbf{u}_m. \quad (60)$$

导弹飞行总时间

$$T_m = \frac{|\mathbf{d}_m|}{v_m}, \quad (61)$$

即导弹在 $t \in [0, T_m]$ 内处于飞行状态，仅需在此时段内通过烟幕遮蔽真目标。

2. 无人机飞行模型

设无人机 FY_k ($k = 1, 2, 3, 4, 5$, 对应 FY1-FY5) 初始位置为 (x_{k0}, y_{k0}, z_{k0}) , 飞行速度 $v_k \in [70, 140]$ m/s, 航向角为 θ_k (平面内与 x 轴夹角, 因等高度飞行, z 坐标保持 z_{k0} 不变)。

无人机在 t 时刻的位置为

$$(x_k(t), y_k(t), z_k(t)) = (x_{k0} + v_k \cdot t \cdot \cos \theta_k, y_{k0} + v_k \cdot t \cdot \sin \theta_k, z_{k0}). \quad (62)$$

若无人机飞行 $t_{k,i}$ 时间后投放第 i 枚烟幕弹 ($i = 1, 2, 3$), 则投放点坐标

$$P_{k,i} = (x_k(t_{k,i}), y_k(t_{k,i}), z_{k0}). \quad (63)$$

3. 干扰弹与遮蔽模型

(1) 干扰弹下落与起爆模型

烟幕弹脱离无人机后仅受重力自由下落, 设重力加速度为 $g = 9.8 \text{ m/s}^2$, 投放点高度为 z_{k0} , 起爆高度为 $z_{k,i}^{\text{det}}$ 。

由自由落体公式 $z_{k0} - z_{k,i}^{\text{det}} = \frac{1}{2}g(t_{k,i}^{\text{fall}})^2$ 推导得到下落时间

$$t_{k,i}^{\text{fall}} = \sqrt{\frac{2(z_{k0} - z_{k,i}^{\text{det}})}{g}}. \quad (64)$$

起爆时刻

$$t_{k,i}^{\text{det}} = t_{k,i} + t_{k,i}^{\text{fall}}, \quad (65)$$

其中 $t_{k,i}$ 为无人机飞行至投放点的时间, 即投放时刻。

(2) 烟幕云团遮蔽模型

烟幕弹起爆后形成半径 $R = 10\text{ m}$ 的球状云团, 以 $v_s = 3\text{ m/s}$ 匀速下沉, 真目标为底面圆心 $(0, 200, 0)$ 、半径 $r = 7\text{ m}$ 、高 $h = 10\text{ m}$ 的圆柱 (空间范围: $x^2 + (y - 200)^2 \leq 7^2$ 且 $0 \leq z \leq 10$)。

起爆后 τ 时刻 ($\tau \geq 0$) 云团中心坐标为

$$C_{k,i}(\tau) = (x_k(t_{k,i}), y_k(t_{k,i}), z_{k,i}^{\text{det}} - v_s \cdot \tau). \quad (66)$$

遮蔽判定条件为云团与真目标空间交集非空, 即存在点 (x, y, z) 满足以下条件:

$$\begin{cases} (x - x_k(t_{k,i}))^2 + (y - y_k(t_{k,i}))^2 + (z - (z_{k,i}^{\text{det}} - v_s \cdot \tau))^2 \leq R^2, \\ x^2 + (y - 200)^2 \leq r^2, \\ 0 \leq z \leq 10. \end{cases} \quad (67)$$

设满足遮蔽条件的 τ 范围为 $[\tau_{k,i}^{\text{start}}, \tau_{k,i}^{\text{end}}]$, 且 $\tau_{k,i}^{\text{end}} \leq 20\text{ s}$ (起爆后 20 s 内有效), 则有效遮蔽时长

$$\Delta t_{k,i} = \tau_{k,i}^{\text{end}} - \tau_{k,i}^{\text{start}}, \quad (68)$$

其中若 $\tau_{k,i}^{\text{start}} > 20$, 则 $\Delta t_{k,i} = 0$ 。

4. 优化目标函数

核心目标为最大化所有烟幕弹对真目标的有效遮蔽总时长, 且遮蔽需与导弹飞行时段重叠 (若遮蔽时段不在任何 $[0, T_m]$ 内, 无实际意义), 即

$$\max F = \sum_{k=1}^5 \sum_{i=1}^{n_k} \Delta t_{k,i} \cdot I(\exists m : [t_{k,i}^{\text{det}} + \tau_{k,i}^{\text{start}}, t_{k,i}^{\text{det}} + \tau_{k,i}^{\text{end}}] \cap [0, T_m] \neq \emptyset), \quad (69)$$

其中, $n_k \leq 3$ 为无人机 FY_k 的投弹数量, $I(\cdot)$ 为指示函数 (条件满足时取 1, 否则取 0)。

5. 约束条件

存在的约束有无人机速度约束、无人机投弹间隔约束、烟幕弹空间有效性约

束以及投弹数量约束，即

$$\begin{cases} 70 \leq v_k \leq 140 & (\forall k = 1, 2, 3, 4, 5), \\ t_{k,i+1} \geq t_{k,i} + 1 & (\forall k = 1, 2, 3, 4, 5; i = 1, 2), \\ \sqrt{x_k(t_{k,i})^2 + (y_k(t_{k,i}) - 200)^2} \leq R + r = 17 \text{ m} & (\forall k, i), \\ n_k \leq 3 & (\forall k = 1, 2, 3, 4, 5). \end{cases} \quad (70)$$

4.5.2 模型的求解

采用智能优化算法对模型进行求解。优化变量涵盖各无人机的飞行速度、航向角及各枚烟幕弹的投放时刻与起爆高度。

如图 20 所示展现了初始场景配置中导弹 (M1、M2、M3)、多架 UAV (UAV1 至 UAV5) 以及假目标、真目标的位置分布，通过三维坐标系 (X、Y、Z 轴) 呈现出各实体的空间位置情况。

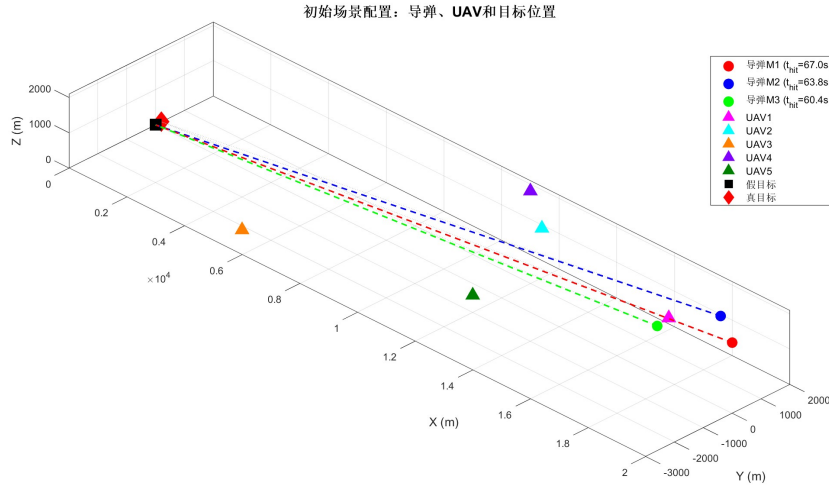


图 20: 导弹、无人机与目标的初始空间位置分布示意图

通过算法迭代搜索最优的无人机协同投弹策略，确保烟幕的有效覆盖时段与各导弹飞行时段充分重叠。该优化模型能够有效调度无人机资源，求得最终总有效遮蔽时长为 10.650 s。进行求解的代码详见支撑材料 Q_5.m，详细方案见支撑材料 result3.xlsx。

5 模型总结

5.1 模型优点

1. 高精度数值求解框架

模型采用了 Newton-Raphson 和 Halley 方法等高阶数值算法，能够精确求解视线遮蔽的进入和离开时刻。相比传统的扫描方法，这种解析-数值混合求解将时间精度提升到了 10^{-10} 秒级别，确保了遮蔽时长计算的准确性。

2. 多层次优化策略设计

模型展现了渐进式的优化架构：粗网格搜索 → 中等精度搜索 → 精细搜索 → 优化器微调。特别是问题三中的 Memetic GA（遗传算法 + 爬山 + 退火）混合策略，有效平衡了全局搜索能力和局部精化效率。

3. 完善的物理建模与动态仿真

模型充分考虑了实际物理过程，包括：(1) 烟幕云团的动态扩散（水平扩散系数、线性膨胀）；(2) 指数趋近终速的下沉模型（更符合空气动力学）；(3) 自由落体弹道计算；(4) 蒙特卡洛仿真验证（考虑参数扰动）。

4. 工程实用性强

代码具有良好的模块化设计，支持并行计算加速（parfor），包含详细的可视化输出功能。特别是对约束条件的处理（如投弹间隔、高度限制等）采用了软惩罚函数（Huber 平滑），避免了优化过程中的数值不稳定。

5.2 模型缺点

1. 计算复杂度较高

随着无人机和烟幕弹数量增加，搜索空间呈指数增长。问题五中 5 架无人机、15 枚烟幕弹的场景需要优化 40 个变量，即使采用了分层搜索和并行计算，仍需要大量计算资源。对于更大规模的实战场景，可能需要进一步的算法优化或近似方法。

2. 模型假设的局限性

虽然考虑了烟幕扩散，但仍存在简化假设：假设烟幕云团为理想球形，忽略了风场影响和非均匀扩散；导弹轨迹假设为直线匀速运动，未考虑实际的制导

修正；未考虑多枚烟幕弹之间的相互作用（如气流干扰）。这些简化可能导致实际应用中的偏差。

参考文献

- [1] 潘志杰. 面向原子动力学蒙特卡洛模拟的压缩存储与计算加速研究. Master's thesis, 杭州电子科技大学, 2025.
- [2] 汤龙. 多参数非线性优化方法关键技术研究及应用. PhD thesis, 湖南大学, 2013.
- [3] 袁彬. 面向昂贵约束优化问题的代理模型辅助差分进化算法研究及其应用. PhD thesis, 南昌大学, 2024.
- [4] 李家睿. 基于高斯混合朴素贝叶斯全局优化 lightgbm 的 x 银行精准营销策略研究. Master's thesis, 广西科技大学, 2024.
- [5] 刘乐. 基于牛顿拉夫逊法的含分布式电源配电网潮流计算. Master's thesis, 河北科技大学, 2018.
- [6] 林惠. 非线性约束不可分非凸优化问题分裂序列二次规划算法研究. Master's thesis, 广西民族大学, 2023.
- [7] 豆包电脑版, 1.66.5_win, 字节跳动, 2025-09-04
- [8] 豆包电脑版, 1.66.5_win, 字节跳动, 2025-09-08
- [9] 豆包电脑版, 1.66.5_win, 字节跳动, 2025-09-07

6 附录 A 支撑材料文件清单

表 3: 支撑材料文件清单

序号	文件名	说明
1	Q_1.m	解决问题一的 MATLAB 代码
2	Q_2.m	解决问题二的 MATLAB 代码
2	Q_3.m	解决问题三的 MATLAB 代码
4	Q_4.m	解决问题四的 MATLAB 代码
5	Q_5.m	解决问题五的 MATLAB 代码
6	result1.xlsx	问题三的结果输出文件
7	result2.xlsx	问题四的结果输出文件
8	result3.xlsx	问题五的结果输出文件
9	AI 工具使用详情.pdf	AI 工具使用详情

7 附录 B 解决问题一的核心 MATLAB 代码

```
1 %% 运动学建模
2 Cf = @(t) C0 + [0, 0, -vs * (t - t0)];
3 Mf = @(t) M10 + VM * t * uh;
4 Mp = VM * uh;
5 Cp = [0, 0, -vs];
6 %% 视线遮蔽判定函数
7 cmd = @(t) cmp(t, Mf, Cf, T);
8 ts = linspace(t0, t0 + DT, 1000);
9 mv = inf;
10 ti = t0;
11 si = 0.5;
12
13 for i = 1:length(ts)
14     t = ts(i);
15     Mt = Mf(t);
```

```

16     Ct = Cf(t);
17     U = Ct - Mt;
18     V = T - Mt;
19     Vn = dot(V, V);
20     ss = dot(U, V) / Vn;
21     if ss > 0 && ss < 1
22         d2 = dot(U, U) - (dot(U, V))^2 / Vn;
23         val = abs(d2 - R^2);
24         if val < mv
25             mv = val;
26             ti = t;
27             si = ss;
28         end
29     end
30 end
31
32 [tin, sin] = n2s(ti, si, Mf, Cf, T, R, Mp, Cp);
33 gf = @(t) norm(Mf(t) - Cf(t))^2 - R^2;
34 gp = @(t) 2 * dot(Mf(t) - Cf(t), Mp - Cp);
35 gpp = 2 * norm(Mp - Cp)^2;
36 ts2 = linspace(tin, t0 + DT, 1000);
37 for i = 2:length(ts2)
38     if gf(ts2(i-1)) < 0 && gf(ts2(i)) > 0
39         to = (ts2(i-1) + ts2(i)) / 2;
40         break;
41     end
42 end
43 tout = hs(to, gf, gp, gpp);
44 Ts = tout - tin;

```

```

45
46 function [dm, ss] = cmp(t, Mf, Cf, T)
47     Mt = Mf(t);
48     Ct = Cf(t);
49     U = Ct - Mt;
50     V = T - Mt;
51     Vn = dot(V, V);
52     ss = dot(U, V) / Vn;
53     if ss <= 0
54         dm = norm(Mt - Ct);
55     elseif ss >= 1
56         dm = norm(T - Ct);
57     else
58         dm = sqrt(dot(U, U) - (dot(U, V))^2 / Vn);
59     end
60 end
61
62 function [t, s] = n2s(t0, s0, Mf, Cf, T, R, Mp, Cp)
63     t = t0;
64     s = s0;
65     mi = 100;
66     t1 = 1e-12;
67     for it = 1:mi
68         Mt = Mf(t);
69         Ct = Cf(t);
70         V = T - Mt;
71         Vp = -Mp;
72         W = Mt + s * V;
73         Rv = W - Ct;

```

```

74         F1 = dot(Rv, Rv) - R^2;
75         F2 = dot(Rv, V);
76         if abs(F1) < t1 && abs(F2) < t1
77             break;
78         end
79         dWdt = (1 - s) * Mp;
80         dRdt = dWdt - Cp;
81         J11 = 2 * dot(Rv, dRdt);
82         J12 = 2 * dot(Rv, V);
83         J21 = dot(dRdt, V) + dot(Rv, Vp);
84         J22 = dot(V, V);
85         dJ = J11 * J22 - J12 * J21;
86         if abs(dJ) < 1e-15
87             break;
88         end
89         dt = -(J22 * F1 - J12 * F2) / dJ;
90         ds = -(-J21 * F1 + J11 * F2) / dJ;
91         t = t + dt;
92         s = s + ds;
93         s = max(1e-10, min(s, 1 - 1e-10));
94         if abs(dt) < t1 && abs(ds) < t1
95             break;
96         end
97     end
98 end
99
100 function t = hs(t0, gf, gpf, gpp)
101     t = t0;
102     mi = 100;

```

```

103     t1 = 1e-12;
104
105     for it = 1:mi
106         g = gf(t);
107         gp = gpf(t);
108
109         if abs(g) < t1
110             break;
111         end
112
113         dn = 2 * gp^2 - g * gpp;
114         if abs(dn) < 1e-15
115             dt = -g / gp;
116         else
117             dt = -2 * g * gp / dn;
118         end
119
120         t = t + dt;
121
122         if abs(dt) < t1
123             break;
124         end
125     end
126 end

```

8 附录 C 解决问题二的核心 MATLAB 代码

```

1 %计算遮蔽时间
2 function st = f(p, f1, m, u, v, t, r, s, g, mx)

```

```

3     th = p(1);
4     vu = p(2);
5     tr = p(3);
6     tf = p(4);
7     %轨道
8     d = [cos(th), sin(th), 0];
9     pr = f1 + vu * tr * d;
10    pb = pr + vu * tf * d - [0, 0, 0.5*g*tf^2];
11    tb = tr + tf;
12    dt = 0.002;
13    ts = tb:dt:(tb + mx);
14    te = -1;
15    tx = -1;
16    for i = 1:length(ts)
17        ti = ts(i);
18        mt = m + v * ti * u;
19        ct = pb;
20        ct(3) = ct(3) - s * (ti - tb);
21        ui = ct - mt;
22        vi = t - mt;
23        vn = sum(vi.^2);
24        if vn < 1e-6
25            break;
26        end
27        si = (ui * vi') / vn;
28        si = max(0, min(1, si));
29        if si <= 0
30            dm = norm(mt - ct);
31        elseif si >= 1

```

```

32         dm = norm(t - ct);
33     else
34         w = mt + si * vi;
35         dm = norm(w - ct);
36     end
37     %是否遮蔽了
38     if dm <= r + 1e-6
39         if te < 0
40             te = ti;
41         end
42         tx = ti;
43     elseif te > 0
44         break;
45     end
46 end
47 if te > 0 && tx > te
48     st = tx - te;
49 else
50     st = 0;
51 end
52 end

```

9 附录 D 解决问题三的核心 MATLAB 代码

```

1 %
    判断点c是否在由点m到点t的线段上，并且点c与线段的距离是否小于等于r
2 function b = f1(m, t, c, r)
3     ab = t - m;
4     ap = c - m;

```



```

5      l2 = dot(ab, ab);
6      if l2 < 1e-12
7          b = (norm(c - m) <= r);
8          return;
9      end
10     s = dot(ap, ab) / l2;
11     s = max(0, min(1, s));
12     q = m + s * ab;
13     b = (norm(c - q) <= r);
14 end
15
16 % 计算云团在给定时间a时的半径
17 function r = f2(p, a)
18     if a <= 0
19         r = p.r0;
20         return;
21     end
22     r = sqrt(max(0, p.r0^2 + 2 * p.dx * a)) + p.al * a;
23     r = min(p.rm, r);
24 end
25
26 % 计算云团在给定时间a时的z轴位置（高度）
27 function z = f3(p, h, a)
28     if a <= 0
29         z = h;
30         return;
31     end
32     z = h - p.vt * (1 - exp(-a / p.ts));
33 end

```

```

34
35 % 评估个体适应度，基于遮蔽时间和其他约束
36 function ft = f4(id, ctx)
37     id = sort(unique(max(1, min(numel(ctx.cd), id)),
38         'ascend')));
39     if numel(id) < 3
40         ft = 1e9;
41         return;
42     end
43     mk = ctx.cd(id(1)).mk | ctx.cd(id(2)).mk |
44         ctx.cd(id(3)).mk;
45     tt = sum(mk) * ctx.p.dt;
46     td = sort([ctx.cd(id).td]);
47     d1 = td(2) - td(1);
48     d2 = td(3) - td(2);
49     pi = h1(ctx.p.mi - d1, ctx.p.hd) + h1(ctx.p.mi - d2,
50         ctx.p.hd);
51     pw = 0;
52     if tt < ctx.p.tl
53         pw = pw + h1(ctx.p.tl - tt, ctx.p.hd);
54     end
55     if tt > ctx.p.th
56         pw = pw + h1(tt - ctx.p.th, ctx.p.hd);
57     end
58     ft = abs(tt - ctx.p.tg) + ctx.p.pi * pi + ctx.p.pw *
59         pw;
60 end
61
62 % 计算Huber平滑惩罚

```

```

59 function y = h1(x, d)
60     if x <= 0
61         y = 0;
62     else
63         if x <= d
64             y = 0.5 * (x / d)^2;
65         else
66             y = x - d / 2;
67         end
68     end
69 end

```

10 附录 E 解决问题四的核心 MATLAB 代码

```

1    %% 核心问题设置
2    uav_init = [17800,0,1800; 12000,1400,1400;
3               6000,-3000,700]; % FY1,FY2,FY3
4    M1_init  = [20000,0,2000];
5    v_M1     = 300;
6    target   = [0,200,0];
7    t_end    = 66.67;
8    g        = 9.8;
9    R        = 10;
10   % 12 维: 3 架 * (theta,v,t0,dt)
11   patternMin = [0,70,0,0];
12   patternMax = [2*pi,140,t_end,t_end];
13   VarMin = repmat(patternMin,1,3);
14   VarMax = repmat(patternMax,1,3);
15   nVar   = numel(VarMin); % 12

```

```

15 %% 核心目标函数
16 function totalTime =
    EvaluateShieldingTime_v3(sol,uav_init,M1_init,v_M1,target,t_end)
17     R = 10; g=9.8;
18     th    = [sol(1),sol(5),sol(9)];
19     v     = [sol(2),sol(6),sol(10)];
20     t0    = [sol(3),sol(7),sol(11)];
21     dt_k  = [sol(4),sol(8),sol(12)];
22     % 严格边界约束
23     for k=1:3
24         v(k)      = clamp(v(k), 70, 140);
25         t0(k)     = clamp(t0(k), 0, t_end-20);
26         dt_k(k)   = clamp(dt_k(k), 0.5, 20);
27         % 确保不超过结束时间
28         if t0(k) + dt_k(k) + 20 > t_end
29             excess = t0(k) + dt_k(k) + 20 - t_end;
30             if dt_k(k) - excess >= 0.5
31                 dt_k(k) = dt_k(k) - excess;
32             else
33                 dt_k(k) = 0.5;
34                 t0(k) = max(0, t_end - 20.5);
35             end
36         end
37     end
38
39     t1 = t0 + dt_k;
40
41     % 计算爆点
42     C0 = zeros(3,3);

```

```

43     for k=1:3
44         vx = v(k)*cos(th(k));
45         vy = v(k)*sin(th(k));
46         exp_x = uav_init(k,1) + vx * t1(k);
47         exp_y = uav_init(k,2) + vy * t1(k);
48         exp_z = uav_init(k,3) - 0.5 * g * (dt_k(k)^2);
49         exp_z = max(0, exp_z); % 确保高度非负
50         C0(k,:) = [exp_x, exp_y, exp_z];
51     end
52
53     % 精确覆盖判定
54     dir = (target - M1_init) / norm(target - M1_init);
55
56     function flag = cov_at(t)
57         M1pos = M1_init + dir * v_M1 * t;
58         vL = target - M1pos;
59         a = dot(vL,vL);
60
61         if a <= 1e-15
62             flag = false;
63             return;
64         end
65
66         for kk=1:3
67             if t >= t1(kk) && t <= t1(kk)+20
68                 dt_smoke = t - t1(kk);
69                 C = C0(kk,:) - [0, 0, 3*dt_smoke];
70                 % 射线与球相交测试
71                 L0 = M1pos - C;

```

```

72         b = 2*dot(L0,vL);
73         c = dot(L0,L0) - R^2;
74         D = b^2 - 4*a*c;
75
76         if D >= -1e-10 % 数值容差
77             if D < 0, D = 0; end
78             sqrtD = sqrt(D);
79             u1 = (-b + sqrtD)/(2*a);
80             u2 = (-b - sqrtD)/(2*a);
81
82             % 检查交点是否在线段上
83             eps = 1e-8;
84             if (u1 >= -eps && u1 <= 1+eps) ||
85                 (u2 >= -eps && u2 <= 1+eps)
86                 flag = true;
87                 return;
88             end
89
90             % 检查线段是否完全在球内
91             if (u1 < -eps && u2 > 1+eps) || (u2
92                 < -eps && u1 > 1+eps)
93                 flag = true;
94                 return;
95             end
96         end
97     end
98     flag = false;
99 end

```

```

99      % 多分辨率扫描
100     dt_coarse = 0.2;
101     dt_fine = 0.01;
102     % 粗扫描
103     times_coarse = 0:dt_coarse:t_end;
104     covered_coarse = false(size(times_coarse));
105     for i=1:numel(times_coarse)
106         covered_coarse(i) = cov_at(times_coarse(i));
107     end
108     if ~any(covered_coarse)
109         totalTime = 0;
110         return;
111     end
112     % 识别潜在区间
113     intervals = [];
114     for i=1:numel(times_coarse)-1
115         if covered_coarse(i) || covered_coarse(i+1)
116             intervals = [intervals; times_coarse(i),
117                           times_coarse(i+1)]; %#ok<AGROW>
118         end
119     end
120     % 合并重叠区间
121     if ~isempty(intervals)
122         intervals = merge_intervals(intervals);
123     end
124     % 精细扫描
125     covered_segments = [];
126     for int_idx = 1:size(intervals,1)
127         t_start = intervals(int_idx,1);

```

```

127         t_end_int = intervals(int_idx,2);
128         times_fine = t_start:dt_fine:t_end_int;
129         covered_fine = false(size(times_fine));
130         for i=1:numel(times_fine)
131             covered_fine(i) = cov_at(times_fine(i));
132         end
133         % 提取覆盖段
134         in_segment = false;
135         seg_start = 0;
136         for i=1:numel(times_fine)
137             if covered_fine(i) && ~in_segment
138                 seg_start = times_fine(i);
139                 in_segment = true;
140             elseif ~covered_fine(i) && in_segment
141                 covered_segments = [covered_segments;
142                                     seg_start, times_fine(i-1)];
143                 %#ok<AGROW>
144                 in_segment = false;
145             end
146         end
147         if in_segment
148             covered_segments = [covered_segments;
149                                 seg_start, times_fine(end)]; %#ok<AGROW>
150         end
151         % 合并并计算总时间
152         if isempty(covered_segments)
153             totalTime = 0;
154         else

```



```

153         covered_segments =
                merge_intervals(covered_segments);
154         totalTime = sum(covered_segments(:,2) -
                covered_segments(:,1));
155     end
156     if totalTime < 0, totalTime = 0; end
157     if totalTime > t_end, totalTime = t_end; end
158 end
159 function merged = merge_intervals(intervals)
160     if isempty(intervals)
161         merged = [];
162         return;
163     end
164     [~,idx] = sort(intervals(:,1));
165     intervals = intervals(idx,:);
166     merged = intervals(1,:);
167     for i=2:size(intervals,1)
168         if intervals(i,1) <= merged(end,2) + 1e-6
169             merged(end,2) = max(merged(end,2),
                intervals(i,2));
170         else
171             merged = [merged; intervals(i,:)];
                %#ok<AGROW>
172         end
173     end
174 end
175 %% 核心优化算法 - PSO
176 function [best_sol, best_val] = core_pso(VarMin, VarMax,
        uav_init, M1_init, v_M1, target, t_end)

```

```

177     nVar = numel(VarMin);
178     pop_size = 50;
179     max_iter = 100;
180     % 初始化种群
181     pop = zeros(pop_size, nVar);
182     for i = 1:pop_size
183         pop(i,:) = VarMin +
184             rand(1,nVar).*(VarMax-VarMin);
185     end
186     % 初始化速度和个体最优
187     vel = zeros(pop_size, nVar);
188     p_best = pop;
189     p_best_val = zeros(pop_size, 1);
190     % 评估初始种群
191     for i = 1:pop_size
192         p_best_val(i) =
193             EvaluateShieldingTime_v3(pop(i,:), uav_init,
194             M1_init, v_M1, target, t_end);
195     end
196     % 全局最优
197     [g_best_val, idx] = max(p_best_val);
198     g_best = pop(idx,:);
199     % PSO 参数
200     w_max = 0.9; w_min = 0.4;
201     c1 = 2; c2 = 2;
202     % 主循环
203     for iter = 1:max_iter
204         w = w_max - (w_max - w_min) * iter / max_iter;
205         for i = 1:pop_size

```

```

203         r1 = rand(1, nVar);
204         r2 = rand(1, nVar);
205         % 更新速度
206         vel(i,:) = w * vel(i,:) + c1 * r1 .*
                (p_best(i,:) - pop(i,:)) + c2 * r2 .*
                (g_best - pop(i,:));
207         % 速度限制
208         vel_max = 0.1 * (VarMax - VarMin);
209         vel(i,:) = max(min(vel(i,:), vel_max),
                -vel_max);
210         % 更新位置
211         pop(i,:) = pop(i,:) + vel(i,:);
212         pop(i,:) = max(min(pop(i,:), VarMax),
                VarMin);
213         % 评估适应度
214         fitness = EvaluateShieldingTime_v3(pop(i,:),
                uav_init, M1_init, v_M1, target, t_end);
215         % 更新个体最优
216         if fitness > p_best_val(i)
217             p_best(i,:) = pop(i,:);
218             p_best_val(i) = fitness;
219         end
220         % 更新全局最优
221         if fitness > g_best_val
222             g_best = pop(i,:);
223             g_best_val = fitness;
224         end
225     end
226     if mod(iter, 20) == 0

```

```

227         fprintf('Iter %d: Best = %.4f\n', iter,
                g_best_val);
228     end
229 end
230 best_sol = g_best;
231 best_val = g_best_val;
232 end
233 %% 辅助函数
234 function y = clamp(x, lo, hi)
235     y = max(lo, min(hi, x));
236 end
237 %% 主运行代码
238 [best_solution, best_fitness] = core_pso(VarMin, VarMax,
        uav_init, M1_init, v_M1, target, t_end);
239 fprintf('最优遮蔽时间: %.4f 秒\n', best_fitness);
240 fprintf('最优解:\n');
241 for i = 1:3
242     idx = (i-1)*4;
243     fprintf('UAV%d: theta=%.3f, v=%.1f, t0=%.1f,
            dt=%.1f\n', ...
244         i, best_solution(idx+1), best_solution(idx+2),
            best_solution(idx+3), best_solution(idx+4));
245 end

```

11 附录 F 解决问题五的核心 MATLAB 代码

```

1 function
    [neg,D]=fobj(x,U0,M0,T0,vm,vdir,g,Rr,Tr,vz,tg,nu,kmax,nm)
2 i=0;

```

```

3     v =x(i+1:i+nu);
4     i=i+nu;
5     ph=x(i+1:i+nu);
6     i=i+nu;
7     td=x(i+1:i+nu*kmax);
8     i=i+nu*kmax;
9     ta=x(i+1:i+nu*kmax);
10    % ----- 约束惩罚项 -----
11    pen=0;
12    for u=1:nu
13        s=(u-1)*kmax+(1:kmax);
14        tdu=td(s); tau=ta(s);
15        tds=sort(tdu);
16        if any(diff(tds)<1),
17            pen=pen+1e3*sum(max(0,1-diff(tds))); end
18        tdet=tdu+tau;
19        pen=pen+1e3*sum(max(0,tdet-tg(end)))+1e3*sum(max(0,-tdu));
20    end
21    % ----- 计算烟雾弹轨迹 -----
22    du=[cos(ph(:)),sin(ph(:)),zeros(nu,1)];
23    B=[];
24    for u=1:nu
25        for k=1:kmax
26            id=(u-1)*kmax+k;
27            if td(id)<0 || ta(id)<=0, continue; end
28            pr=U0(u,:)+v(u)*td(id)*du(u,:);
29            pb=pr+v(u)*ta(id)*du(u,:)+[0,0,-0.5*g*ta(id)^2];
30            tb=td(id)+ta(id);

```

```

31         B=[B; pb,tb,tb+Tr]; % (x,y,z,t_burst,t_end)
32     end
33 end
34
35 % ----- 计算每枚导弹被遮蔽时间 -----
36 cv=zeros(nm,1);
37 for m=1:nm
38     m0=M0(m,:); vd=vdir(m,:);
39     hit=norm(m0)/vm;
40     vis=false(size(tg));
41     for it=1:numel(tg)
42         t=tg(it); if t>hit, break; end
43         mp=m0+vm*t*vd;
44         ok=false;
45         for b=1:size(B,1)
46             tb=B(b,4); te=B(b,5);
47             if t<tb||t>te, continue; end
48             c=B(b,1:3)+[0,0,vz*(t-tb)];
49             if dseg(c,mp,T0)<=Rr, ok=true; break; end
50         end
51         vis(it)=ok;
52     end
53     cv(m)=sum(vis)*(tg(2)-tg(1));
54 end
55
56 % ----- 目标函数 -----
57 tot=sum(cv);
58 neg=-(tot)+pen;
59 D.summary=table((1:nm)',cv,'VariableNames',{'Missile','Coverage_s'}

```

```
60 end
61
62 function d=dseg(c,a,b)
63     ab=b-a; ac=c-a;
64     t=dot(ac,ab)/(dot(ab,ab)+eps);
65     t=max(0,min(1,t));
66     p=a+t*ab;
67     d=norm(c-p);
68 end
```